

Computational Physics

Lecture 4: Numerical Derivatives

Finite differences

- forward and backward differences
- central difference
- higher-order differences

Automatic differentiation

$$\frac{df}{dx} \simeq \frac{f(x+h) - f(x)}{h}$$

References:

Computational Physics by Mark Newman, Chapter 5

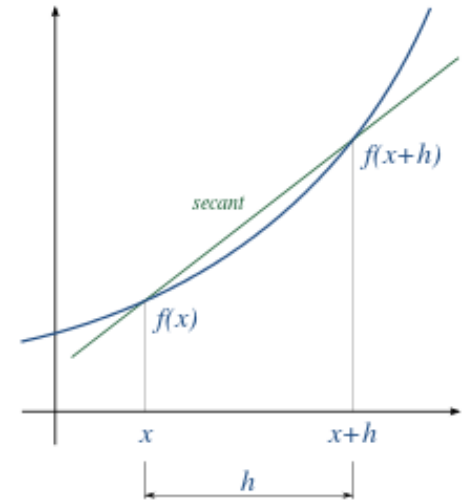
<https://websites.umich.edu/~mejn/cp2/chapters/chapter5.pdf>

Numerical differentiation

Let's now turn to the problem of evaluating the derivative:

$$\frac{df}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

We need numerical differentiation e.g. when function f is known only at a discrete set of points



Forward and backward differences

The definition basically gives the practical implementation for small finite h

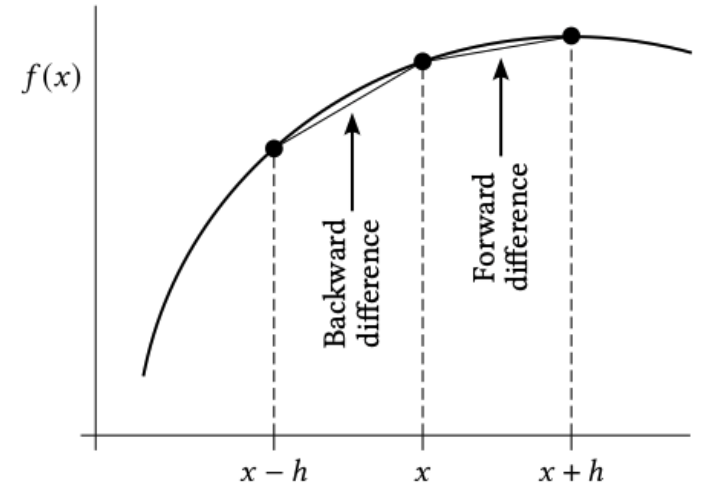
$$\frac{df}{dx} \simeq \frac{f(x+h) - f(x)}{h}$$

forward difference
(slope going towards positive x)

The mirror image definition is

$$\frac{df}{dx} \simeq \frac{f(x) - f(x-h)}{h}$$

backward difference



give similar estimates, but one can be preferred e.g. when there is a discontinuity on one side, or want derivative at a boundary

How to choose h ?



Let's look at errors

Errors on forward and backward differences

Take Taylor expansion $f(x + h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \dots$

Rearranging we have $f'(x) = \underbrace{\frac{f(x+h) - f(x)}{h}}_{\text{forward difference}} - \underbrace{\frac{1}{2}hf''(x)}_{\text{approximation error (linear in h)}} + \dots$

Note however that we are subtracting $f(x+h)$ and $f(x)$, which are very close. If h becomes very small we can get a very large rounding error (remember exercise on accuracy error)

Given machine precision $\varepsilon \sim 10^{-16}$ the smallest choice for h is such that for $f(x+h) - f(x)$ the rounding error is $2\varepsilon|f(x)|$ (the worst case is the sum of the two rounding errors), so

$$\frac{2\varepsilon|f(x)|}{h} \quad \text{rounding error}$$

Numerical differentiation: errors on forward and backward differences

The total error for the derivative $\delta = \frac{2\epsilon|f(x)|}{h} + \frac{1}{2}h|f''(x)|$.

To obtain **the value of h that minimize this error** we differentiate on h and set it to zero:

$$-\frac{2\epsilon|f(x)|}{h^2} + \frac{1}{2}|f''(x)| = 0, \quad \longrightarrow \quad h = \sqrt{4\epsilon \left| \frac{f(x)}{f''(x)} \right|}.$$

Substituting in δ : $\delta = h|f''(x)| = \sqrt{4\epsilon|f(x)f''(x)|}$.

If e.g. $f(x)$ and $f''(x)$ are of order 1, the minimum h will be roughly $\sqrt{\epsilon} \sim 10^{-8}$, so half the numerical precision, and the minimo error $\delta \sim 10^{-8}$

How to improve accuracy?

Central difference

A simple improvement is using the **central difference**

$$\frac{df}{dx} \simeq \frac{f(x+h) - f(x-h)}{2h}$$

the two points are now placed symmetrically around x

Let's compute again the approximation error

$$\begin{aligned} f(x + \tfrac{1}{2}h) &= f(x) + \tfrac{1}{2}hf'(x) + \tfrac{1}{8}h^2f''(x) + \tfrac{1}{48}h^3f'''(x) + \dots \\ f(x - \tfrac{1}{2}h) &= f(x) - \tfrac{1}{2}hf'(x) + \tfrac{1}{8}h^2f''(x) - \tfrac{1}{48}h^3f'''(x) + \dots \end{aligned}$$

Subtracting the second expression from the first and rearranging for $f'(x)$

$$f'(x) = \frac{f(x + \tfrac{1}{2}h) - f(x - \tfrac{1}{2}h)}{h} - \boxed{\tfrac{1}{24}h^2f'''(x)} + \dots$$

The error is one order in h better

approximation
error of order h^2

Central difference

If we add the rounding error the total error is $\delta = \frac{2\epsilon|f(x)|}{h} + \frac{1}{24}h^2|f'''(x)|$.

with optimal h value $h = \left(24\epsilon\left|\frac{f(x)}{f'''(x)}\right|\right)^{1/3}$,

Substituted in δ gives the optimal error

$$\delta = \frac{1}{8}h^2|f'''(x)| = \left(\frac{9}{8}\epsilon^2[f(x)]^2|f'''(x)|\right)^{1/3}.$$

If now $f(x)$ and $f''(x)$ are of order 1, minimum h will be roughly $\epsilon^{1/3} \sim 10^{-5}$ (higher than forward/backward differences), but the error δ will be $\epsilon^{2/3} \sim 10^{-10}$, so smaller.

In the same situation the central difference is indeed more accurate than the forward and backward differences, by a factor of 100 or so in this case, although we use a larger value of h

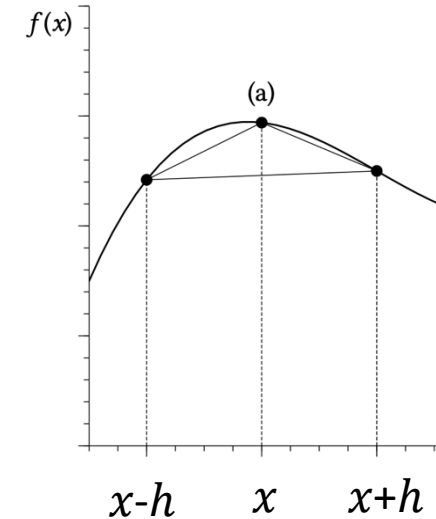
Derivative of a sampled function

Suppose we have access to values of $f(x)$ only at regularly spaced sample points a distance h apart

Suppose we want the derivative in (a)

We can use forward, backward difference with an interval of h . For the central difference we can only use $x+h$ and $x-h$, making the interval $2h$

$$\frac{df}{dx} \simeq \frac{f(x+h) - f(x-h)}{2h}$$



So it is not always true the central difference is better, only if $\delta_{\text{centr}} < \delta_{\text{for}}$ (for δ_{centr} use $2h$ instead of h)

$$\frac{1}{2}h^2 |f'''(x)| < h |f''(x)| \quad \longrightarrow \quad h < 2 \left| \frac{f''(x)}{f'''(x)} \right|.$$

Example with forward and central differentiation

Given $f(x) = 1 + \frac{1}{2} \tanh(2x)$, $x \in [-2, 2]$ with $f'(x) = \operatorname{sech}^2(2x) = 1/\cosh^2(2x)$

- 1) Sample $f(x)$ over evenly h -spaced points. Choose e.g. $h = 10^{-1}$. Compute for this grid the forward and central differences (use $2h$ for the central one).
- 2) Compare on a graph the analytical and numerical derivatives (try also changing h to see how they vary)
- 3) Consider e.g. $x_0 = 0.5$, and compute in this case the errors on forward and central differences (just subtract the estimated $f'(x)$ from its actual value). Plot these errors as a function of h , using $h = 10^{-1}, 10^{-2}, \dots, 10^{-16}$. Comment on the results and the trends

Example with forward and central differentiation

```
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    """f(x) = 1 + 0.5 * tanh(2x)."""
    return 1.0 + 0.5 * np.tanh(2.0 * x)

def f_exact(x):
    """Analytic derivative f'(x) = 1 / cosh^2(2x)."""
    return 1.0 / np.cosh(2.0 * x) ** 2

h = 1e-1
x = np.arange(-2.0, 2.0 + h / 2, h)
n = len(x)

fx = np.zeros(n)
for i in range(n):
    fx[i] = f(x[i])

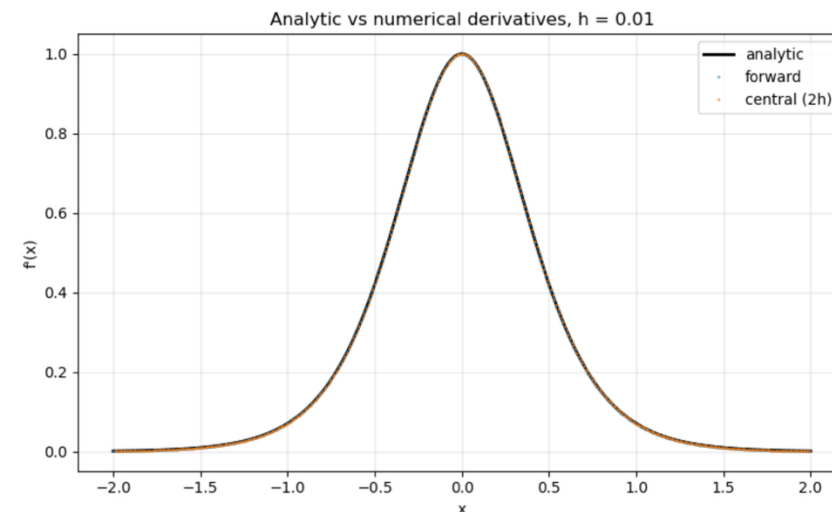
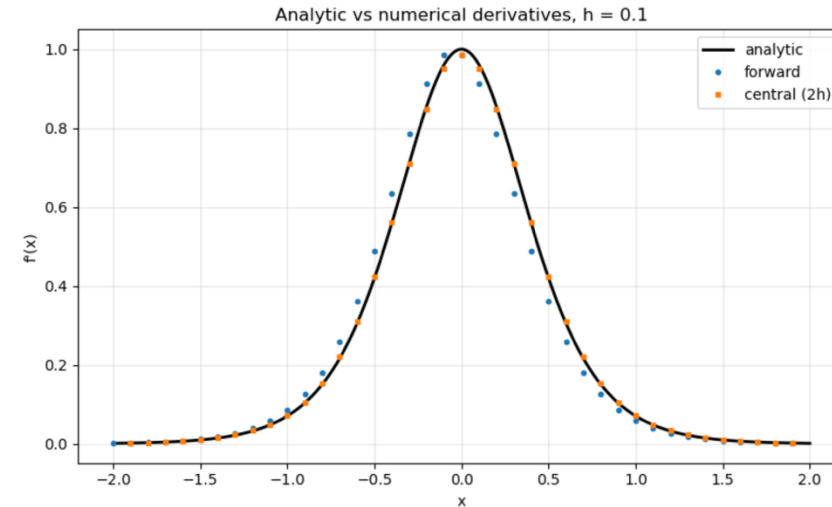
d_fwd = np.zeros(n - 1)
d_cen = np.zeros(n - 2)

for i in range(n - 1):
    d_fwd[i] = (fx[i + 1] - fx[i]) / h

for i in range(1, n - 1):
    d_cen[i - 1] = (fx[i + 1] - fx[i - 1]) / (2.0 * h)

x_plot = np.linspace(-2.0, 2.0, 400)

plt.figure(figsize=(8, 5))
plt.plot(x_plot, f_exact(x_plot), "--", lw=2, color="black", label="analytic")
plt.plot(x[:-1], d_fwd, "o", ms=3, label="forward")
plt.plot(x[1:-1], d_cen, "s", ms=3, label="central (2h)")
plt.xlabel("x")
plt.ylabel("f'(x)")
plt.title(f"Analytic vs numerical derivatives, h = {h:g}")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



Example with forward and central differentiation

```
#error comparison
x0 = 0.5
exact = f_exact(x0)
hs = [10.0 ** (-k) for k in range(1, 17)]

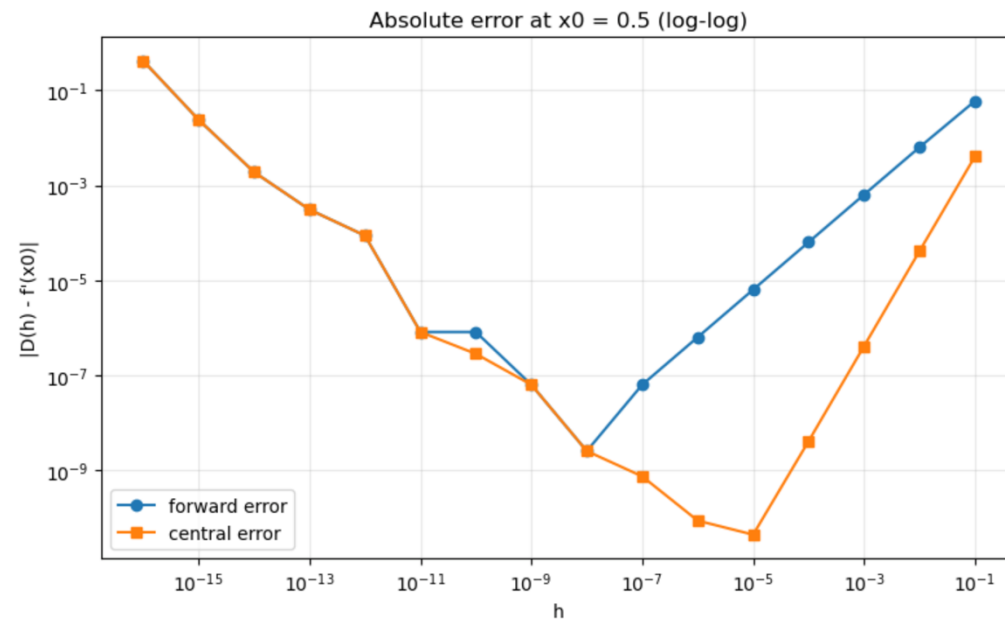
err_fwd = []
err_cen = []

for h in hs:
    f_left = f(x0 - h)
    f_mid = f(x0)
    f_right = f(x0 + h)

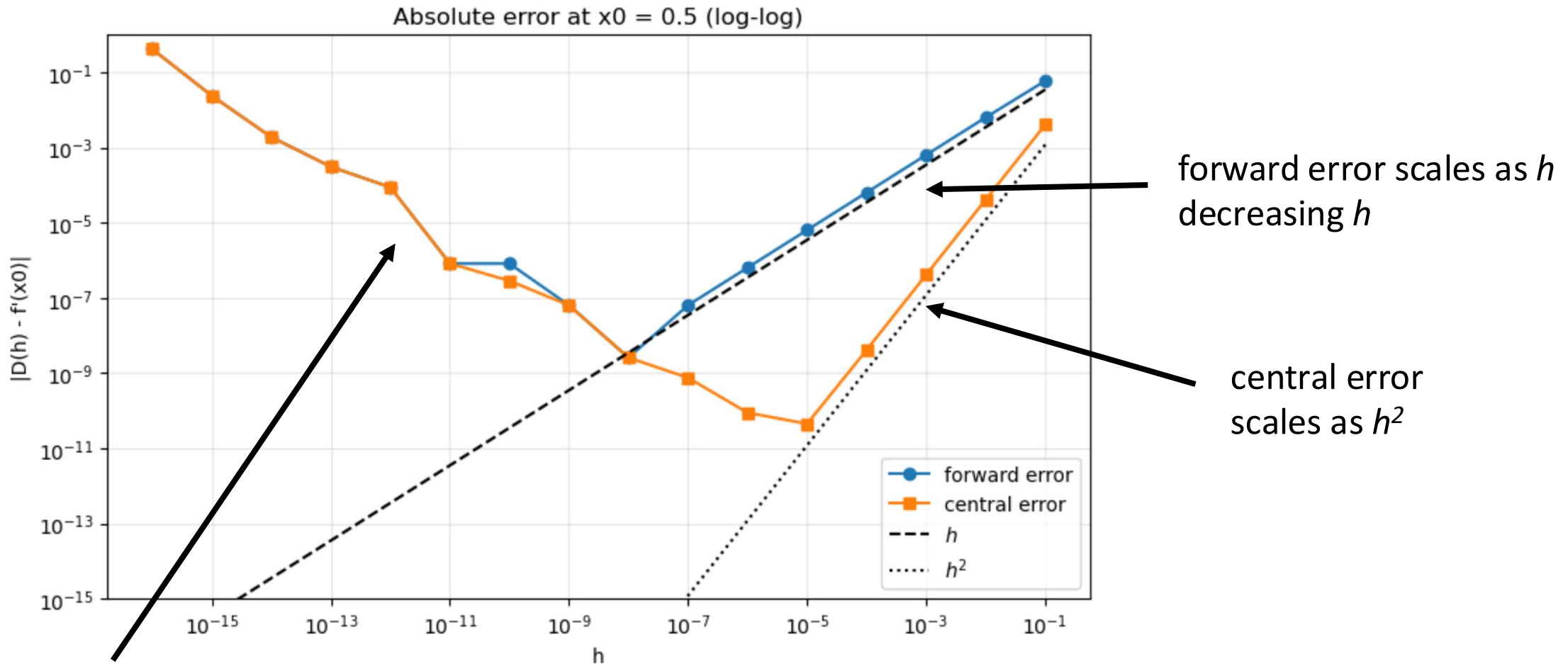
    d_plus = (f_right - f_mid) / h
    d_cent = (f_right - f_left) / (2.0 * h)

    e_plus = abs(d_plus - exact)
    e_cent = abs(d_cent - exact)
    err_fwd.append(e_plus)
    err_cen.append(e_cent)

plt.figure(figsize=(8, 5))
plt.loglog(hs, err_fwd, "o-", label="forward error")
plt.loglog(hs, err_cen, "s-", label="central error")
plt.xlabel("h")
plt.ylabel("|D(h) - f'(x0)|")
plt.title(f"Absolute error at x0 = {x0} (log-log)")
plt.legend()
plt.grid(True, which="both", alpha=0.3)
plt.tight_layout()
plt.savefig("parteC_errore.png", dpi=160)
plt.show()
```



Example with forward and central differentiation



High-order central difference

Up to now we did the same as the trapezoidal rule, i.e. connecting two dots with a straight line. But we can think of using quadratic or higher-order polynomials

Consider a quadratic curve: $y = ax^2 + bx + c$

Make the quadratic curve equal to $f(x)$ at $x=-h$, 0, and $+h$ giving conditions for a, b, c :

$$ah^2 - bh + c = f(-h), \quad c = f(0), \quad ah^2 + bh + c = f(h),$$

Compute the derivative of $y(x)$ in the fitted point $x=0$: $\frac{dy}{dx} = [2ax + b]_{x=0} = b$.

So we need only the parameter b to compute the derivative in $x=0$:

$$\begin{array}{l} \text{subtract} \end{array} \quad \begin{array}{l} ah^2 - bh + c = f(-h), \\ ah^2 + bh + c = f(h), \end{array} \quad \rightarrow \quad 2bh = f(h) - f(-h) \quad \rightarrow \quad \frac{df}{dx} \simeq \frac{f(h) - f(-h)}{2h}$$

High-order central difference

The same result applies to any other point, just sliding the function on the x-axis to put this point in 0:

$$\frac{df}{dx} \simeq \frac{f(x+h) - f(x-h)}{2h} \quad \text{the quadratic approximation is just the central difference!}$$

Can still improve using higher order polynomials, using same procedure and increasing number of points.
Depending on order need to choose points differently

Cubic (degree 3): 4 points around center. $x = -\frac{3}{2}h, -\frac{1}{2}h, \frac{1}{2}h, \frac{3}{2}h$

$$f'(x) \approx \frac{f\left(x - \frac{3h}{2}\right) - 27f\left(x - \frac{h}{2}\right) + 27f\left(x + \frac{h}{2}\right) - f\left(x + \frac{3h}{2}\right)}{24h}.$$

Quartic (degree 4): 5 points, included center $x=2h, -h, 0, h$, and $2h$

$$f'(x) \approx \frac{f(x-2h) - 8f(x-h) + 8f(x+h) - f(x+2h)}{12h}.$$

High-order central difference

Table of coefficients

Degree	$f(-\frac{5}{2}h)$	$f(-2h)$	$f(-\frac{3}{2}h)$	$f(-h)$	$f(-\frac{1}{2}h)$	$f(0)$	$f(\frac{1}{2}h)$	$f(h)$	$f(\frac{3}{2}h)$	$f(2h)$	$f(\frac{5}{2}h)$	Error
1					-1		1					$O(h^2)$
2				$-\frac{1}{2}$				$\frac{1}{2}$				$O(h^2)$
3			$\frac{1}{24}$		$-\frac{27}{24}$		$\frac{27}{24}$		$-\frac{1}{24}$			$O(h^4)$
4		$\frac{1}{12}$		$-\frac{2}{3}$				$\frac{2}{3}$		$-\frac{1}{12}$		$O(h^4)$
5	$-\frac{3}{640}$		$\frac{25}{384}$		$-\frac{75}{64}$		$\frac{75}{64}$		$-\frac{25}{384}$		$\frac{3}{640}$	$O(h^6)$

At each order the expression is a linear combination of the samples, divided by h

Error must be balanced again against rounding error. If ε is machine precision and the truncation error is of order $O(h^n)$, then h should not be much smaller than $h \sim \sqrt[n+1]{\varepsilon}$:

The higher the finite difference order is, the larger h should be

The coefficient for $f(0)$ in all the approximations is zero.

Second derivatives

To estimate $f''(x)$ we need $f'(x)$, which is typically itself estimated. Use central difference for $f''(x)$:

$$f''(x) \simeq \frac{f'(x + \frac{1}{2}h) - f'(x - \frac{1}{2}h)}{h} \quad \text{with } f' \text{ estimated with central difference in } x+h/2 \text{ and } x-h/2$$
$$f'(x + \frac{1}{2}h) \simeq \frac{f(x+h) - f(x)}{h},$$
$$f'(x - \frac{1}{2}h) \simeq \frac{f(x) - f(x-h)}{h}.$$

$$f''(x) \simeq \frac{[f(x+h) - f(x)]/h - [f(x) - f(x-h)]/h}{h} = \boxed{\frac{f(x+h) - 2f(x) + f(x-h)}{h^2}}.$$

simplest approximation for the second derivative

To compute error start from
Taylor expansion and sum them

$$f(x+h) = f(x) + hf'(x) + \frac{1}{2}h^2 f''(x) + \frac{1}{6}h^3 f'''(x) + \frac{1}{24}f''''(x) + \dots$$
$$f(x-h) = f(x) - hf'(x) + \frac{1}{2}h^2 f''(x) - \frac{1}{6}h^3 f'''(x) + \frac{1}{24}f''''(x) - \dots$$

$$f''(x) = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2} - \boxed{\frac{1}{12}h^2 f''''(x)} + \dots$$

error

Second derivatives

Combining with rounding error: each term contributes with $2\epsilon|f(x)|$ on each value of $f(x)$, giving $4\epsilon|f(x)|/h^2$

$$\delta = \frac{4\epsilon|f(x)|}{h^2} + \frac{1}{12}h^2|f''''(x)|.$$

With the optimum value of h and δ : $h = \left(48\epsilon \left|\frac{f(x)}{f''''(x)}\right|\right)^{1/4}$ $\delta = \frac{1}{6}h^2|f''''(x)| = \left(\frac{4}{3}\epsilon|f(x)f''''(x)|\right)^{1/2}.$

With $f(x)$ and $f''''(x)$ of order 1, the error is of order $\sqrt{\epsilon} \sim 10^{-8}$, same as forward difference. There are other higher orders approximations.

Going to the third derivative, we would find

$$f'''(x) \simeq \frac{f(x + \frac{3}{2}h) - 3f(x + \frac{1}{2}h) + 3f(x - \frac{1}{2}h) - f(x - \frac{3}{2}h)}{h^3}.$$

General formula: $f^{(n)}(x) = \frac{1}{h^n} \sum_{k=0}^n (-1)^k \binom{n}{k} f[x + (n/2 - k)h] + O(h^2)$

Partial derivatives

Let us have a function of two variables: $f(x,y)$. Use central difference to calculate **first- and second-order partial derivatives** in x and y

$$f_x(x, y) \approx \frac{f(x + h, y) - f(x - h, y)}{2h}$$

$$f_y(x, y) \approx \frac{f(x, y + k) - f(x, y - k)}{2k}$$

$$f_{xx}(x, y) \approx \frac{f(x + h, y) - 2f(x, y) + f(x - h, y)}{h^2}$$

$$f_{yy}(x, y) \approx \frac{f(x, y + k) - 2f(x, y) + f(x, y - k)}{k^2}$$

$$f_{xy}(x, y) \approx \frac{f(x + h, y + k) - f(x + h, y - k) - f(x - h, y + k) + f(x - h, y - k)}{4hk}.$$

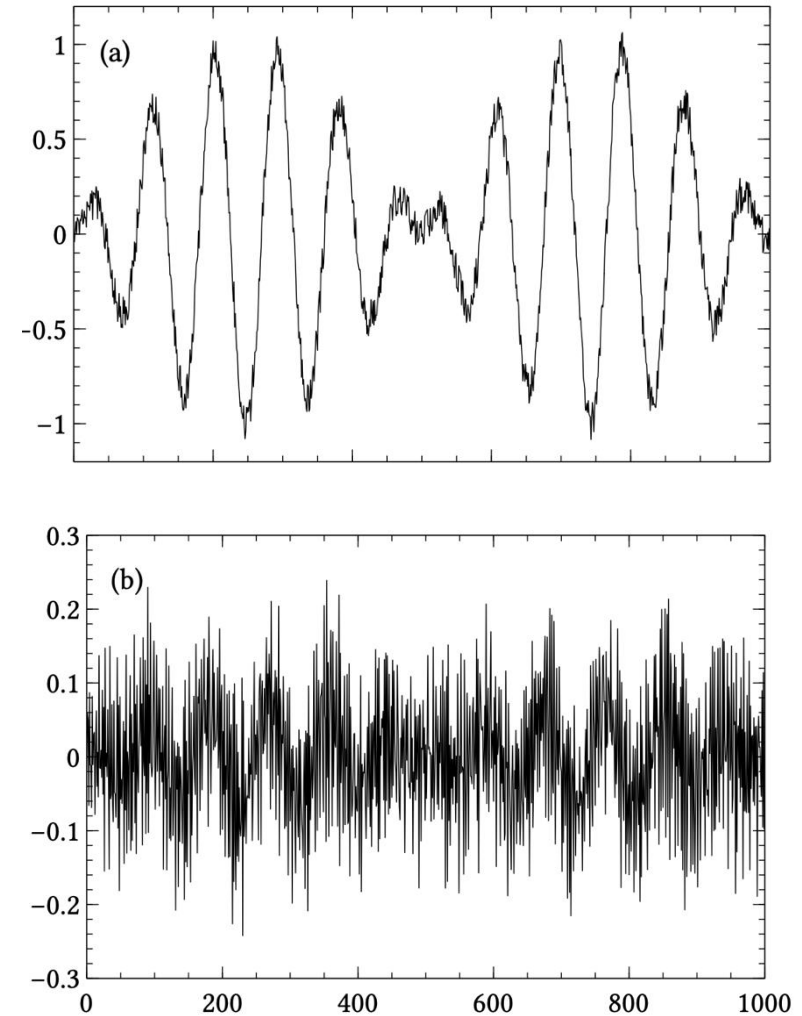
Derivatives of noisy data

The typical situation is having derivatives of noisy data, from some numerical or experimental measurement

The overall shape is clear, but the curve is not smooth

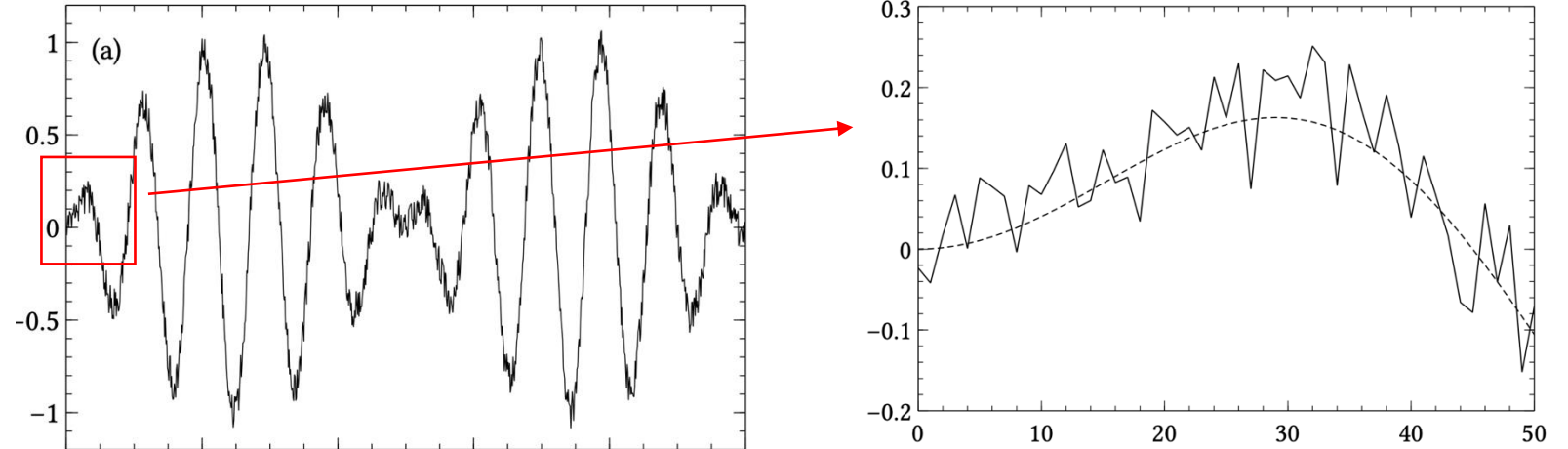
Suppose we compute the derivative with forward difference

We have now a much worse noise problem



Derivatives of noisy data

The reason is clear zooming in



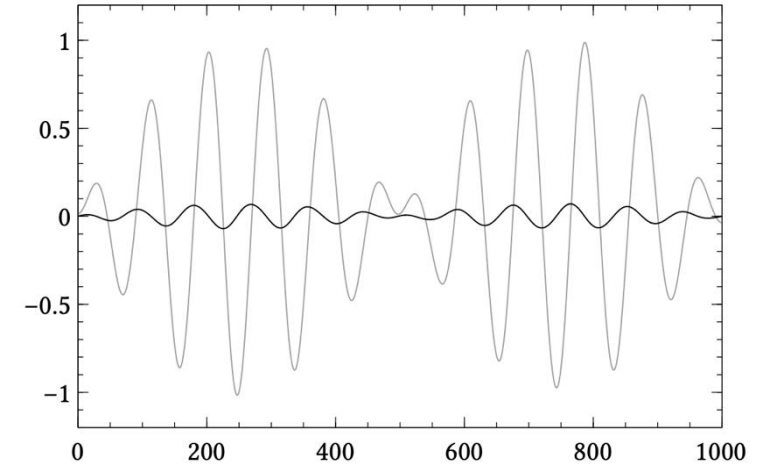
The solid line represents the actual data, and the dotted line what the curve should look like (we usually don't know the latter)

The noise make some very steep slopes, which generate large spurious derivatives

How to improve?

Derivatives of noisy data

- 1) Increase h : find an optimal value of h that balance noise against approximation of derivative, giving it a similar role to the rounding error
- 2) fit a curve to a portion of the data near the point where we want the derivative, then differentiate the curve
- 3) smooth the data in some other manner:
use the Fourier transform, or perform a running average, i.e. for each point you substitute the average over N points before and N points afterward. Works if the curve does not change much in the $2N$ range.



Automatic differentiation

Automatic differentiation (or **algorithmic differentiation**) is a computational technique to evaluate the exact derivative of a specified function. Used e.g. in machine learning

Start with a function $x(t)$ and replace it with the couple $[x(t), x'(t)]$, where x is the value of the variable at t and x' is its derivative with respect to t at the same point.

E.g. for $x = t^2$ at $t = 1$ store $[1, 2]$, corresponding to $x = t^2 = 1$ and $x' = 2t = 2$

Define elementary mathematical operations for these objects

Examples of unary operators:

multiplication with constant $x \rightarrow cx$ becomes $[x, x'] \rightarrow [cx, cx']$.
taking a power $x \rightarrow x^c$ becomes $[x, x'] \rightarrow [x^c, cx^{c-1}x']$ using $\frac{d}{dt}(x^c) = cx^{c-1} \frac{dx}{dt}$

Examples of binary operators:

addition $x, y \rightarrow x+y$ becomes $[x, x'], [y, y'] \rightarrow [x+y, x'+y']$
multiplication $x, y \rightarrow xy$ becomes $[x, x'], [y, y'] \rightarrow [xy, xy' + yx']$ using $\frac{d}{dt}(xy) = x \frac{dy}{dt} + y \frac{dx}{dt}$.

Automatic differentiation

Common operations:

$$x \rightarrow x + c \quad \text{becomes} \quad [x, x'] \rightarrow [x + c, x']$$

$$x \rightarrow cx \quad \text{becomes} \quad [x, x'] \rightarrow [cx, cx']$$

$$x \rightarrow x^c \quad \text{becomes} \quad [x, x'] \rightarrow [x^c, cx^{c-1}x']$$

$$x \rightarrow c^x \quad \text{becomes} \quad [x, x'] \rightarrow [c^x, (c^x \ln c) x']$$

$$x \rightarrow \log x \quad \text{becomes} \quad [x, x'] \rightarrow [\log x, x'/x]$$

$$x \rightarrow \sin x \quad \text{becomes} \quad [x, x'] \rightarrow [\sin x, x' \cos x]$$

$$x \rightarrow \cos x \quad \text{becomes} \quad [x, x'] \rightarrow [\cos x, -x' \sin x]$$

$$x \rightarrow \tan x \quad \text{becomes} \quad [x, x'] \rightarrow [\tan x, x' \sec^2 x]$$

$$x, y \rightarrow x + y \quad \text{becomes} \quad [x, x'], [y, y'] \rightarrow [x + y, x' + y']$$

$$x, y \rightarrow x - y \quad \text{becomes} \quad [x, x'], [y, y'] \rightarrow [x - y, x' - y']$$

$$x, y \rightarrow xy \quad \text{becomes} \quad [x, x'], [y, y'] \rightarrow [xy, xy' + yx']$$

$$x, y \rightarrow \frac{x}{y} \quad \text{becomes} \quad [x, x'], [y, y'] \rightarrow \left[\frac{x}{y}, \frac{yx' - xy'}{y^2} \right]$$

$$x, y \rightarrow x^y \quad \text{becomes} \quad [x, x'], [y, y'] \rightarrow [x^y, (x^y \log x)x' + yx^{y-1}y']$$

Automatic differentiation

General rule for unary operations $U(x)$ is

$$x \rightarrow U(x) \quad \text{becomes} \quad [x, x'] \rightarrow [U(x), U'(x) x'],$$

for binary operations $B(x,y)$

$$x, y \rightarrow B(x, y) \quad \text{becomes} \quad [x, x'], [y, y'] \rightarrow [B(x, y), B_x(x, y) x' + B_y(x, y) y'],$$

where B_x and B_y denote the partial derivatives of B with respect to x and y .

We can now combine elementary operations to calculate any more complicated function that can be expressed as a sequence of such operations

Finally, to retrieve the value of a derivative $f'(t)$ at a specific value t , start from the couple $[t, 1]$ and apply $f(t)$: you will get following the general rule of unary operators

$$[t, 1] \quad \text{becomes} \quad [f(t), t' f'(t)] = [f(t), f'(t)]$$

so the output will be the value of f and f' in t

Automatic differentiation: example

Compute at $t=0.5$ the value of the derivative of $f(x) = 3x^2 + e^x$.

Decompose into the following functions to perform in succession:

$u_1 = x^2,$	$[x, x'] \rightarrow [x^c, cx^{c-1}x']$
$u_2 = 3u_1,$	$[x, x'] \rightarrow [cx, cx']$
$u_3 = e^x,$	$[x, x'] \rightarrow [c^x, (c^x \ln c) x']$
$f(x) = u_2 + u_3.$	$[x, x'], [y, y'] \rightarrow [x + y, x' + y']$

Define now for each function appearing, the generalized operation for the couple which includes the derivative

Call now the function f (defined in terms of a pair) using as argument $[0.5,1]$ to compute $f'(0.5)$.

Automatic differentiation: Example

```
from math import exp
```

```
# Define elementary mathematical operations
```

```
def multiply(x,c):
```

```
    vx,dx = x
```

```
    return [c*vx,c*dx]
```

$$[x, x'] \rightarrow [cx, cx']$$

```
def power(x,c):
```

```
    vx,dx = x
```

```
    return [vx**c,c*vx**(c-1)*dx]
```

$$[x, x'] \rightarrow [x^c, cx^{c-1}x']$$

```
def exponential(x):
```

```
    vx,dx = x
```

```
    return [exp(vx),exp(vx)*dx]
```

$$[x, x'] \rightarrow [c^x, (c^x \ln c) x']$$

```
def add(x,y):
```

```
    vx,dx = x
```

```
    vy,dy = y
```

```
    return [vx+vy,dx+dy]
```

$$[x, x'], [y, y'] \rightarrow [x + y, x' + y']$$

```
# Compute the function f(x) = 3*x**2 + exp(x)
```

```
def f(x):
```

```
    u1 = power(x,2)
```

```
    u2 = multiply(u1,3)
```

```
    u3 = exponential(x)
```

```
    return add(u2,u3)
```



Combination of elementary functions to obtain $f(x), f'(x)$

```
# Use this function to calculate f(t) and its derivative at t=1$
```

```
t = [0.5,1]
```

```
print(f(t))
```

```
[2.398721270700128, 4.648721270700128]
```

Each function has in input a pair of values $x = [\text{value}, \text{derivative}]$, and outputs the operation on the couple again as a pair of values

Calling $f(x)$ with argument $[t,1]$ to compute the derivative at t . Here $t=0.5$. Returns $[f(t), f'(t)]$

Automatic differentiation

For this method there is no approximation involved except the rounding error

Can also be extended to the calculation of higher derivatives.

E.g. for second derivatives x gets replaced by a three-valued triple $[x, x', x'']$

$$x \rightarrow U(x) \quad \text{becomes} \quad [x, x', x''] \rightarrow [U(x), U'(x) x', U''(x) (x')^2 + U'(x) x''],$$

$$x, y \rightarrow B(x, y) \quad \text{becomes} \quad [x, x', x''], [y, y', y''] \rightarrow [B(x, y), B_x(x, y) x' + B_y(x, y) y', \\ B_{xx}(x, y) (x')^2 + 2B_{xy}(x, y) x' y' + B_{yy}(x, y) (y')^2 + B_x(x, y) x'' + B_y(x, y) y''],$$

B_x and B_y are first derivatives of $B(x, y)$ with respect to its two arguments and B_{xx} , B_{xy} , and B_{yy} are second derivatives.

Automatic differentiation: Implementation

Many packages implement this method: TensorFlow, PyTorch, JAX.

Python:

- **JAX:** <https://docs.jax.dev/>
- **MyGrad:** <https://mygrad.readthedocs.io/en/latest/>
- **TensorFlow, PyTorch, ...**

C++:

- **autodiff:** <https://autodiff.github.io/>
- **xad:** <https://auto-differentiation.github.io/>
- ...

Automatic differentiation: Example with JAX

```
import numpy as np
import jax.numpy as jnp
from jax import grad
from jax import jvp

#function and derivative (for comparison with automatic differentiation)
def f(x):
    return x**3 - 2*x**2 + x - 1
def df(x):
    return 3 * x**2 - 4 * x + 1

# Autodiff derivative forward mode
def dfdx_auto_forward(func, x):
    x_val = jnp.array(x)
    dx = jnp.array(1.0)
    y, dy = jvp(func, (x_val,), (dx,))
    return dy

print(f"{'x':>3s} {'f(x)':>10s} {'df/dx_analyt':>15s} {'df/dx_ad_forw':>15s}")
x_values = np.arange(0, 3, 0.2)
for x in x_values:
    print(f"{'x':>3.1f} {'f(x)':>10.4f} {'df(x)':>15.4f} {'dfdx_auto_forward(f,x)':>15.4f}")
```